

2026-06-03 - SPI Communication

Goal

Use Zephyr's SPI API to communicate with an external SPI device — reading and writing registers over a full-duplex bus, with a practical example using a W25Q flash chip (widely available and cheap).

What you will learn

- How to configure an SPI bus and chip-select in the Device Tree
- How to perform SPI read and write transfers using `spi_transceive`
- How to read a JEDEC ID from an SPI flash as a “hello world” for SPI
- How to write and read back a page of data

Overview

SPI is a synchronous, full-duplex 4-wire bus: MOSI, MISO, SCK, and CS. Zephyr's `spi_transceive` API sends and receives simultaneously. The Device Tree declares which SPI controller and CS pin to use.

app.overlay

```
&spi1 {
    status = "okay";
    cs-gpios = <&gpio0 17 GPIO_ACTIVE_LOW>;

    w25q: w25q@0 {
        compatible = "jedec,spi-nor";
        reg = <0>;
        spi-max-frequency = <8000000>;
        size = <0x800000>; /* 8 Mbit */
        has-dpd;
        t-enter-dpd = <3000>;
        t-exit-dpd = <3000>;
    };
};
```

prj.conf

```
CONFIG_SPI=y
CONFIG_FLASH=y
CONFIG_FLASH_MAP=y
CONFIG_LOG=y
```

Reading the JEDEC ID (raw SPI transaction)

```
#include <zephyr/drivers/spi.h>

static const struct device *spi =
DEVICE_DT_GET(DT_NODELABEL(spi1));

static const struct spi_config cfg = {
    .frequency = 8000000,
    .operation = SPI_OP_MODE_MASTER | SPI_WORD_SET(8),
    .cs = SPI_CS_CONTROL_INIT(DT_NODELABEL(w25q), 2),
};

void read_jedec_id(void)
{
    uint8_t tx_buf[] = { 0x9F };    /* JEDEC Read ID command */
    uint8_t rx_buf[4] = { 0 };

    struct spi_buf tx[] = {{ .buf = tx_buf, .len =
sizeof(tx_buf) }};
    struct spi_buf rx[] = {{ .buf = rx_buf, .len =
sizeof(rx_buf) }};
    struct spi_buf_set tx_set = { .buffers = tx, .count = 1 };
    struct spi_buf_set rx_set = { .buffers = rx, .count = 1 };

    spi_transceive(spi, &cfg, &tx_set, &rx_set);
    /* rx_buf[1..3] = Manufacturer, Memory Type, Capacity */
    printk("JEDEC ID: %02X %02X %02X\n", rx_buf[1], rx_buf[2],
rx_buf[3]);
}
```

Using the Zephyr Flash API (higher level)

```
#include <zephyr/drivers/flash.h>

const struct device *flash_dev =
DEVICE_DT_GET(DT_NODELABEL(w25q));
uint8_t data[256];

flash_read(flash_dev, 0, data, sizeof(data));
flash_erase(flash_dev, 0, 4096);    /* erase one sector
```

```
*/  
    flash_write(flash_dev, 0, data, sizeof(data));
```

nRF52840 DK SPI wiring (W25Q flash module)

Signal	nRF52840 DK pin
MOSI	P0.20
MISO	P0.21
SCK	P0.19
CS	P0.17
VCC	3.3V
GND	GND

Why this matters

SPI is used for displays, flash memory, sensors (IMUs, barometers), DACs, and ADCs. The same `spi_transceive` API works for all of them.

Practice tasks

1. Wire a W25Q16 and confirm the JEDEC ID reads EF 40 15.
2. Erase sector 0, write your name as ASCII, read it back.
3. Try increasing the SPI clock frequency and confirm communication stays stable.

Example folder

See `src/2026-06-03_SPI_Communication` for the day's code.

Next topic

Tomorrow we read analog signals using the nRF52840's built-in ADC.